

Doctor Appointment - How it works

Home notes + DESIGN.md + source code | compact print

In one line - Doctor opens slots -> patient searches -> books -> if taken, waitlist -> cancel promotes next patient.

Story - how this LLD works

Doctor Curious (cardiologist) opens 9:30, 12:30, 16:00. Shubh books 12:30. Kunal tries the same slot and gets waitlisted. Shubh cancels - Kunal is auto-booked. Search can sort slots by start time or by doctor rating (strategy).

Who does what

Doctor: Has specialization, rating, and availability map: slot -> free/busy.

Patient: Person booking.

Booking: Links patient + doctor + slot.

BookingService: search, book, cancel, waitlist promotion.

DoctorSlot (DTO): Temporary search result (doctor + slot) - not saved as its own table row.

SlotRankStrategy: Sort search results (by time or by rating).

Waitlist: FIFO queue per doctor-slot key inside BookingRepository.

Main pieces

Doctor

- spec, rating
- availability map

BookingService

- search / book / cancel

Waitlist deque

- per doctor-slot
- FIFO promote

Rank strategy

- by time
- by rating

Step-by-step at runtime

1. Doctor registers and declareAvailability(['9:30','12:30',...]).
2. Patient searches CARDIOLOGIST with StartTimeRankStrategy - sees free slots sorted.
3. Shubh book(Curious, 12:30): slot free -> create Booking, mark availability False.
4. Kunal book same slot: already False -> add Kunal to waitlist, raise error message.
5. Shubh cancel: mark slot True, delete booking, pop waitlist -> book(Kunal) automatically.
6. Kunal now has the 12:30 booking.

Why this design (plain words)

Why Strategy on search? - Same list of slots, different sort - by time vs rating - without rewriting search.

Why waitlist queue? - Fair order: first waiting patient gets the slot when it frees.

Why DTO DoctorSlot? - Search needs doctor+slot together; we don't want to fake-save that pair.

Remember

- Slots are strings like '12:30' - no date in this demo (Mon vs Tue look the same).
- Booked slots disappear from search (only free ones shown).
- Re-declare availability on a booked slot could wrongly mark it free - careful.

OOP in this project

Encapsulation - Waitlist lives inside repository; services use add/pop helpers.

Abstraction - rank(slots) hides sorting details.

Inheritance - StartTime and Rating strategies share one interface.

Polymorphism - search calls strategy.rank without knowing which sort.

DESIGN.md - full file

Complete markdown from lld/doctors_appointment/DESIGN.md (Q&A for learning).

Full DESIGN.md (same file as in this folder) - DESIGN.md

Designing a Doctor's Appointment System - LLD Q&A

Run: `python -m lld.doctors_appointment.main`

Patterns: Strategy · Repository · DTO · FIFO waitlist

Q1. What problem are we solving?

A. Design a doctor appointment booking system that:
 Registers doctors (specialization + rating) and patients
 Lets doctors declare available time slots
 Lets patients search open slots by specialization
 Ranks results via a pluggable strategy (start time or rating)
 Books a slot (marks unavailable)
 On conflict, adds patient to a FIFO waitlist
 On cancel, frees the slot and auto-promotes the next waitlisted patient

Q2. What are the core entities?

- Entity - Responsibility
- Doctor - Name, specialization, rating, availability: dict[slot -> bool]
- Patient - Name + id
- Booking - patient_id, doctor_id, slot
- DoctorSlot (DTO) - Transient search result: doctor + slot
- BookingService - Search, book, cancel, waitlist promotion
- Availability: True = open, False = booked.

Q3. How is the package structured?

```
doctors_appointment/
|-- model/      # Doctor, Patient, Booking
|-- dto/        # DoctorSlot (search result)
|-- strategy/   # StartTimeRank, RatingBasedRank
|-- repository/ # Doctor, Patient, Booking (+ waitlist)
|-- service/    # DoctorService, PatientService, BookingService
|-- exception/  # Doctor/Patient/Booking not found
|-- enums/      # Specialization
|-- utils/      # "12:30" -> time parsing
```

Q4. Why Strategy for ranking?

A. Search results can be sorted differently without changing search logic.

| Strategy | Sort key |

|-----|-----|

| StartTimeRankStrategy | Slot time ascending |

| RatingBasedRankStrategy | Doctor rating descending |

Strategy is passed at call time (not constructor) so the same session can rank by time then by rating.

```
booking_service.search(Specialization.CARDIOLOGIST, StartTimeRankStrategy())
```

No factory here - caller chooses explicitly.

Q5. What is the book + waitlist + cancel flow?

```
book(patient, doctor, "12:30")
-> if slot open: create Booking, mark False, done
-> if taken: add_to_waitlist("{doctor_id}-{slot}"), raise error

cancel(booking_id)
-> mark slot True, delete booking
-> pop_from_waitlist(...)
-> if next patient: book(next, doctor, slot) # auto-promote
```

Demo: Shubh books Curious @ 12:30 -> Kunal waitlisted -> Shubh cancels -> Kunal auto-booked.

Waitlist uses collections.deque (FIFO) keyed by "{doctor_id}-{slot}".

Q6. Why a DTO (`DoctorSlot`)?

A. Search returns a composite of doctor + slot that is never persisted. Booking is the persisted record. Keeps query shape separate from storage.

Q7. Why three services?

- Service - Owns
- DoctorService - Register + declare availability
- PatientService - Register + lookup
- BookingService - Search / book / cancel - orchestrates all repos

- Avoids one god-class service.

Q8. How do you extend this design?

- Add... - How
- New ranking (fee, distance) - Implement SlotRankStrategy
- Date-aware slots - Replace str slot with date+time value object
- Concurrency - Lock per doctor-slot or DB unique constraint
- Better errors - Replace RuntimeError with domain exceptions
- Single source of truth - Derive availability from bookings only

Q9. Common interview follow-ups

Q. Dual source of truth?

Doctor.availability + BookingRepository both track booked state. Could derive open slots from declared slots minus bookings.

Q. Patient conflict check?

Only same slot string across bookings - patient could book "12:30" with two doctors (no date).

Q. Slots have no date?

"12:30" Monday vs Tuesday are indistinguishable - intentional demo limit.

Q. Strategy at call time vs constructor?

Call-time is more flexible when ranking preference changes per search.

Source code - lld/doctors_appointment/

19 Python files below (compact font to save pages). Read with the notes: find the class name from the cast, then open that file here.

- Tip: start at main.py, then service/, then model/ + strategy/.

FILE: main.py (59 lines)

```
1| from .enums.specialization import Specialization
2| from .repository.booking_repository import BookingRepository
3| from .repository.doctor_repository import DoctorRepository
4| from .repository.patient_repository import PatientRepository
5| from .service.booking_service import BookingService
6| from .service.doctor_service import DoctorService
7| from .service.patient_service import PatientService
8| from .strategy.start_time_rank_strategy import StartTimeRankStrategy
9|
10|
11| def main() -> None:
12|     doctor_repository = DoctorRepository()
13|     patient_repository = PatientRepository()
14|     booking_repository = BookingRepository()
15|
16|     doctor_service = DoctorService(doctor_repository)
17|     patient_service = PatientService(patient_repository)
18|     booking_service = BookingService(
19|         booking_repository, doctor_repository, patient_repository
20|     )
21|     rank_strategy = StartTimeRankStrategy()
22|
23|     curious = doctor_service.register("Curious", Specialization.CARDIOLOGIST, 4.5)
24|     dreadful = doctor_service.register("Dreadful", Specialization.CARDIOLOGIST, 3.8)
25|     doctor_service.register("Daring", Specialization.DERMATOLOGIST, 4.2)
26|
27|     doctor_service.declare_availability(curious.id, ["9:30", "12:30", "16:00"])
28|     doctor_service.declare_availability(dreadful.id, ["12:30", "13:00"])
29|
30|     p1 = patient_service.register("Shubh")
31|     p2 = patient_service.register("Kunal")
32|
33|     print("Available Cardiologist slots:")
34|     slots = booking_service.search(Specialization.CARDIOLOGIST, rank_strategy)
35|     for slot in slots:
36|         print(f'{slot.doctor.name} - {slot.slot}')
37|
38|     b1 = booking_service.book(p1.id, curious.id, "12:30")
39|
40|     print("\nDoctor Curious bookings:")
41|     for booking in booking_service.view_bookings_by_doctor(curious.id):
42|         patient_name = patient_service.find_by_id(booking.patient_id).name
43|         print(f'Booking: Patient ID {patient_name}, Slot {booking.slot}')
44|
45|     try:
46|         booking_service.book(p2.id, curious.id, "12:30")
47|     except Exception as e:
48|         print(f'\nPatient 2 waitlisted: {e}')
49|
50|     booking_service.cancel(b1.id)
51|
52|     print("\nDoctor Curious bookings:")
53|     for booking in booking_service.view_bookings_by_doctor(curious.id):
54|         patient_name = patient_service.find_by_id(booking.patient_id).name
55|         print(f'Booking: Patient ID {patient_name}, Slot {booking.slot}')
56|
57|
58| if __name__ == "__main__":
59|     main()
```

FILE: service/booking_service.py (83 lines)

```
1| from dataclasses import dataclass
2| from uuid import UUID
3|
4| from ..dto.doctor_slot import DoctorSlot
5| from ..enums.specialization import Specialization
6| from ..exception.booking_not_found_exception import BookingNotFoundException
7| from ..model.booking import Booking
8| from ..repository.booking_repository import BookingRepository
9| from ..repository.doctor_repository import DoctorRepository
10| from ..repository.patient_repository import PatientRepository
11| from ..strategy.slot_rank_strategy import SlotRankStrategy
12|
13|
14| @dataclass
15| class BookingService:
16|     booking_repo: BookingRepository
17|     doctor_repo: DoctorRepository
18|     patient_repo: PatientRepository
19|
20|     def search(self, spec: Specialization, strategy: SlotRankStrategy) -> list[DoctorSlot]:
21|         doctors = self.doctor_repo.find_by_specialization(spec)
22|         result: list[DoctorSlot] = []
23|
24|         for doctor in doctors:
25|             for slot, is_available in doctor.availability.items():
26|                 if is_available:
27|                     result.append(DoctorSlot(doctor=doctor, slot=slot))
28|
29|         return strategy.rank(result)
30|
31|     def book(self, patient_id: UUID, doctor_id: UUID, slot: str) -> Booking:
```

```

32|     doctor = self.doctor_repo.find_by_id(doctor_id)
33|     availability = doctor.availability
34|
35|     if slot not in availability:
36|         raise RuntimeError(
37|             "Invalid slot: Doctor has not declared availability for this slot."
38|         )
39|
40|     for booking in self.booking_repo.find_by_patient(patient_id):
41|         if booking.slot == slot:
42|             raise RuntimeError("Patient already has an appointment at this time")
43|
44|     if availability[slot]:
45|         booking = Booking(patient_id=patient_id, doctor_id=doctor_id, slot=slot)
46|         self.booking_repo.save(booking)
47|         availability[slot] = False
48|
49|         patient = self.patient_repo.find_by_id(patient_id)
50|         print(
51|             f"\n{patient.name} booked a slot successfully for slot : {slot}"
52|         )
53|
54|         return booking
55|
56|     key = f"{doctor_id}-{slot}"
57|     self.booking_repo.add_to_waitlist(key, patient_id)
58|     raise RuntimeError("Slot already booked. Added to waitlist.")
59|
60| def cancel(self, booking_id: UUID) -> None:
61|     booking = self.booking_repo.get_booking_by_id(booking_id)
62|     if booking is None:
63|         raise BookingNotFoundException("Booking not found")
64|
65|     doctor = self.doctor_repo.find_by_id(booking.doctor_id)
66|     doctor.availability[booking.slot] = True
67|     self.booking_repo.delete(booking)
68|
69|     patient = self.patient_repo.find_by_id(booking.patient_id)
70|     print(
71|         f"\n{patient.name} cancelled the booking for slot : {booking.slot}"
72|     )
73|
74|     key = f"{doctor.id}-{booking.slot}"
75|     next_patient = self.booking_repo.pop_from_waitlist(key)
76|     if next_patient is not None:
77|         self.book(next_patient, doctor.id, booking.slot)
78|
79| def view_bookings_by_doctor(self, doctor_id: UUID) -> list[Booking]:
80|     return self.booking_repo.find_by_doctor(doctor_id)
81|
82| def view_bookings_by_patient(self, patient_id: UUID) -> list[Booking]:
83|     return self.booking_repo.find_by_patient(patient_id)

```

FILE: service/doctor_service.py (24 lines)

```

1| from dataclasses import dataclass
2| from uuid import UUID
3|
4| from ..enums.specialization import Specialization
5| from ..exception.doctor_not_found_exception import DoctorNotFoundException
6| from ..model.doctor import Doctor
7| from ..repository.doctor_repository import DoctorRepository
8|
9|
10| @dataclass
11| class DoctorService:
12|     repo: DoctorRepository
13|
14|     def register(self, name: str, spec: Specialization, rating: float) -> Doctor:
15|         doctor = Doctor(name=name, specialization=spec, rating=rating)
16|         self.repo.save(doctor)
17|         return doctor
18|
19|     def declare_availability(self, doctor_id: UUID, slots: list[str]) -> None:
20|         doc = self.repo.find_by_id(doctor_id)
21|         if doc is None:
22|             raise DoctorNotFoundException("Doctor not found")
23|         for slot in slots:
24|             doc.availability[slot] = True

```

FILE: service/patient_service.py (22 lines)

```

1| from dataclasses import dataclass
2| from uuid import UUID
3|
4| from ..exception.patient_not_found_exception import PatientNotFoundException
5| from ..model.patient import Patient
6| from ..repository.patient_repository import PatientRepository
7|
8|
9| @dataclass
10| class PatientService:
11|     repo: PatientRepository
12|
13|     def register(self, name: str) -> Patient:
14|         patient = Patient(name=name)
15|         self.repo.save(patient)
16|         return patient
17|
18|     def find_by_id(self, patient_id: UUID) -> Patient:
19|         patient = self.repo.find_by_id(patient_id)

```

```

20|     if patient is None:
21|         raise PatientNotFoundException("Patient not found")
22|     return patient

```

FILE: model/booking.py (10 lines)

```

1| from dataclasses import dataclass, field
2| from uuid import UUID, uuid4
3|
4|
5| @dataclass
6| class Booking:
7|     patient_id: UUID
8|     doctor_id: UUID
9|     slot: str
10|     id: UUID = field(default_factory=uuid4)

```

FILE: model/doctor.py (13 lines)

```

1| from dataclasses import dataclass, field
2| from uuid import UUID, uuid4
3|
4| from ..enums.specialization import Specialization
5|
6|
7| @dataclass
8| class Doctor:
9|     name: str
10|     specialization: Specialization
11|     rating: float
12|     id: UUID = field(default_factory=uuid4)
13|     availability: dict[str, bool] = field(default_factory=dict)

```

FILE: model/patient.py (8 lines)

```

1| from dataclasses import dataclass, field
2| from uuid import UUID, uuid4
3|
4|
5| @dataclass
6| class Patient:
7|     name: str
8|     id: UUID = field(default_factory=uuid4)

```

FILE: strategy/rating_based_rank_strategy.py (8 lines)

```

1| from ..dto.doctor_slot import DoctorSlot
2| from .slot_rank_strategy import SlotRankStrategy
3|
4|
5| class RatingBasedRankStrategy(SlotRankStrategy):
6|     def rank(self, slots: list[DoctorSlot]) -> list[DoctorSlot]:
7|         slots.sort(key=lambda slot: slot.doctor.rating, reverse=True)
8|         return slots

```

FILE: strategy/slot_rank_strategy.py (9 lines)

```

1| from abc import ABC, abstractmethod
2|
3| from ..dto.doctor_slot import DoctorSlot
4|
5|
6| class SlotRankStrategy(ABC):
7|     @abstractmethod
8|     def rank(self, slots: list[DoctorSlot]) -> list[DoctorSlot]:
9|         pass

```

FILE: strategy/start_time_rank_strategy.py (9 lines)

```

1| from ..dto.doctor_slot import DoctorSlot
2| from ..utils.utils import convert_string_to_local_time
3| from .slot_rank_strategy import SlotRankStrategy
4|
5|
6| class StartTimeRankStrategy(SlotRankStrategy):
7|     def rank(self, slots: list[DoctorSlot]) -> list[DoctorSlot]:
8|         slots.sort(key=lambda slot: convert_string_to_local_time(slot.slot))
9|         return slots

```

FILE: repository/booking_repository.py (37 lines)

```

1| from collections import deque
2| from dataclasses import dataclass, field
3| from uuid import UUID
4|
5| from ..model.booking import Booking
6|
7|
8| @dataclass
9| class BookingRepository:
10|     booking_map: dict[UUID, Booking] = field(default_factory=dict)
11|     waitlist: dict[str, deque[UUID]] = field(default_factory=dict)
12|
13|     def save(self, booking: Booking) -> None:
14|         self.booking_map[booking.id] = booking
15|
16|     def delete(self, booking: Booking) -> None:
17|         del self.booking_map[booking.id]
18|
19|     def get_booking_by_id(self, booking_id: UUID) -> Booking | None:
20|         return self.booking_map.get(booking_id)
21|
22|     def find_by_doctor(self, doctor_id: UUID) -> list[Booking]:
23|         return [b for b in self.booking_map.values() if b.doctor_id == doctor_id]
24|

```

```

25| def find_by_patient(self, patient_id: UUID) -> list[Booking]:
26|     return [b for b in self.booking_map.values() if b.patient_id == patient_id]
27|
28| def add_to_waitlist(self, doctor_slot_key: str, patient_id: UUID) -> None:
29|     if doctor_slot_key not in self.waitlist:
30|         self.waitlist[doctor_slot_key] = deque()
31|     self.waitlist[doctor_slot_key].append(patient_id)
32|
33| def pop_from_waitlist(self, doctor_slot_key: str) -> UUID | None:
34|     queue = self.waitlist.get(doctor_slot_key)
35|     if queue is None:
36|         return None
37|     return queue.popleft() if queue else None

```

FILE: repository/doctor_repository.py (23 lines)

```

1| from dataclasses import dataclass, field
2| from uuid import UUID
3|
4| from ..enums.specialization import Specialization
5| from ..model.doctor import Doctor
6|
7|
8| @dataclass
9| class DoctorRepository:
10|     doctor_map: dict[UUID, Doctor] = field(default_factory=dict)
11|
12|     def save(self, doctor: Doctor) -> None:
13|         self.doctor_map[doctor.id] = doctor
14|
15|     def find_by_id(self, doctor_id: UUID) -> Doctor | None:
16|         return self.doctor_map.get(doctor_id)
17|
18|     def find_by_specialization(self, specialization: Specialization) -> list[Doctor]:
19|         return [
20|             doc
21|             for doc in self.doctor_map.values()
22|             if doc.specialization == specialization
23|         ]

```

FILE: repository/patient_repository.py (15 lines)

```

1| from dataclasses import dataclass, field
2| from uuid import UUID
3|
4| from ..model.patient import Patient
5|
6|
7| @dataclass
8| class PatientRepository:
9|     patient_map: dict[UUID, Patient] = field(default_factory=dict)
10|
11|     def save(self, patient: Patient) -> None:
12|         self.patient_map[patient.id] = patient
13|
14|     def find_by_id(self, patient_id: UUID) -> Patient | None:
15|         return self.patient_map.get(patient_id)

```

FILE: dto/doctor_slot.py (9 lines)

```

1| from dataclasses import dataclass
2|
3| from ..model.doctor import Doctor
4|
5|
6| @dataclass
7| class DoctorSlot:
8|     doctor: Doctor
9|     slot: str

```

FILE: enums/specialization.py (8 lines)

```

1| from enum import Enum
2|
3|
4| class Specialization(Enum):
5|     CARDIOLOGIST = "CARDIOLOGIST"
6|     DERMATOLOGIST = "DERMATOLOGIST"
7|     ORTHOPEDIC = "ORTHOPEDIC"
8|     GENERAL_PHYSICIAN = "GENERAL_PHYSICIAN"

```

FILE: exception/booking_not_found_exception.py (3 lines)

```

1| class BookingNotFoundException(Exception):
2|     def __init__(self, message: str) -> None:
3|         super().__init__(message)

```

FILE: exception/doctor_not_found_exception.py (3 lines)

```

1| class DoctorNotFoundException(Exception):
2|     def __init__(self, message: str) -> None:
3|         super().__init__(message)

```

FILE: exception/patient_not_found_exception.py (3 lines)

```

1| class PatientNotFoundException(Exception):
2|     def __init__(self, message: str) -> None:
3|         super().__init__(message)

```

FILE: utils/utils.py (6 lines)

(cont.) utils/utils.py

```
1| from datetime import time
2|
3|
4| def convert_string_to_local_time(value: str) -> time:
5|     parts = value.split(":")
6|     return time(hour=int(parts[0]), minute=int(parts[1]))
```